TETRIS

Automated Design Space Exploration of Randomness-Latency Trade-offs in Masked Hardware

Nilotpola Sarma Tapish Patidar Nupur Brahamanya Chandan Karfa

Indian Institute of Technology Guwahati, India

16:30 hrs 12th October 2026, Royal Seginus Hotel, Antalya, Türkiye

Outline

Background

Problem & Motivation

TETRIS Flow

DSE Algorithms

Asymmetric Gadget Packing

COMAR and randomness re-use

Results

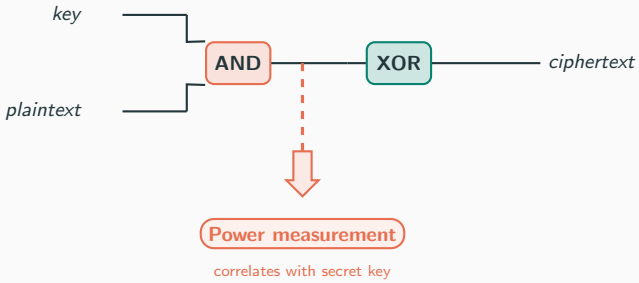
Security

Conclusion

Background

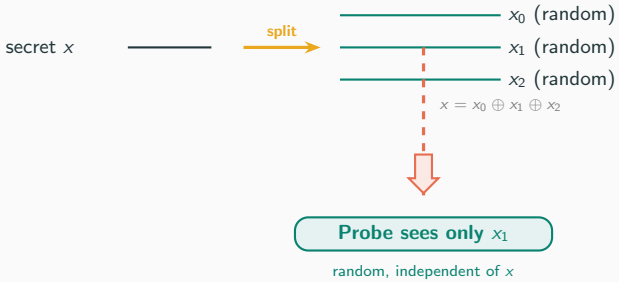
The Attacker's View

An unmasked cipher leaks through power consumption

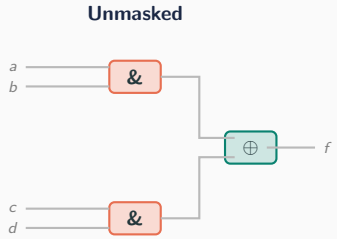


The Attacker's View

Masking: split every wire into d random shares

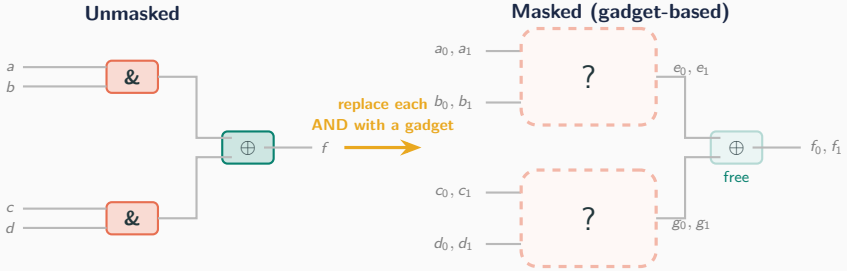


Gadgets: Modular Masked Building Blocks



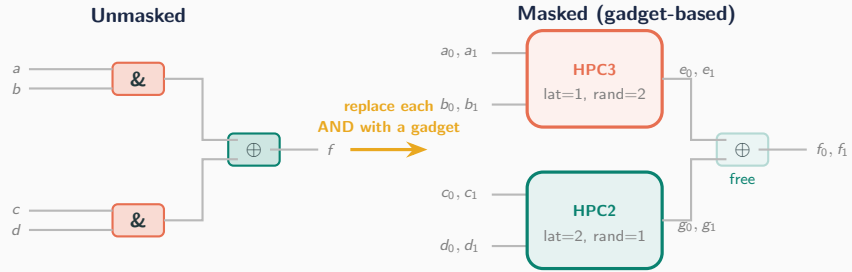
lat denotes max no of regs on a path from input to output.

Gadgets: Modular Masked Building Blocks



lat denotes max no of regs on a path from input to output.

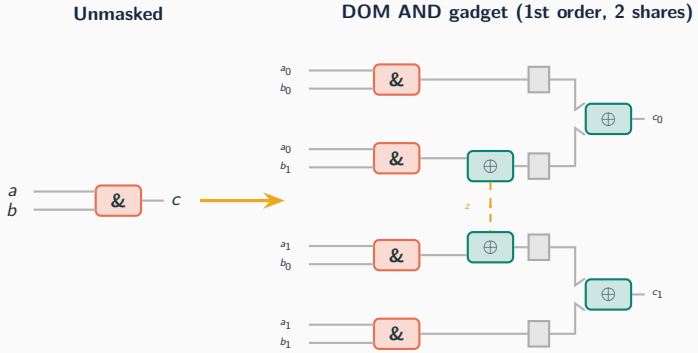
Gadgets: Modular Masked Building Blocks



This assignment (HPC3, HPC2) costs **3 random bits**, latency **2**. A different choice of gadgets costs differently.

lat denotes max no of regs on a path from input to output.

Gadgets: From an AND Gate to the DOM AND Gadget

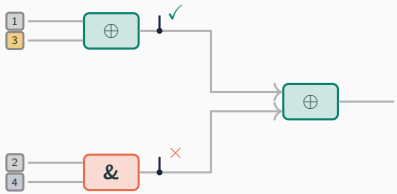


Each share pair (a_0, a_1 and b_0, b_1) is uniformly random and recombines to the true value; every AND becomes 4 sub-ANDs plus a fresh re-mask z on the two cross-domain terms. **Not all 4 registers play the same role.**

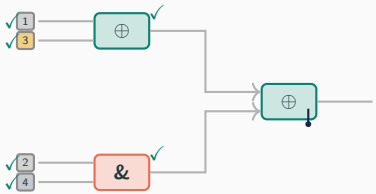
Why registers? Glitches Affect Masked Hardware

- **Glitches**: gates settle at different times, so a combinational circuit can transiently compute an unintended function of *several shares at once* – a probe during a glitch sees more than the final value

Naive: unsafe under glitches



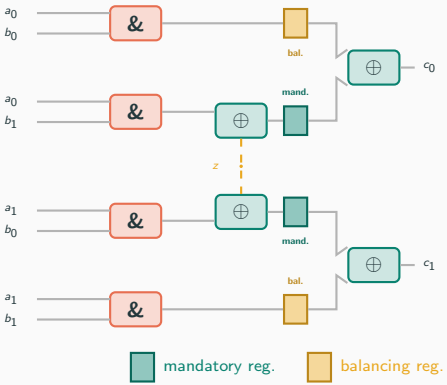
Masked: safe under glitches



1 / 2 share of a secret 3 random mask 4 public/known value × invalid probe position ✓ valid probe position

Gadgets: Glitch-Stopping vs Pipelining Registers

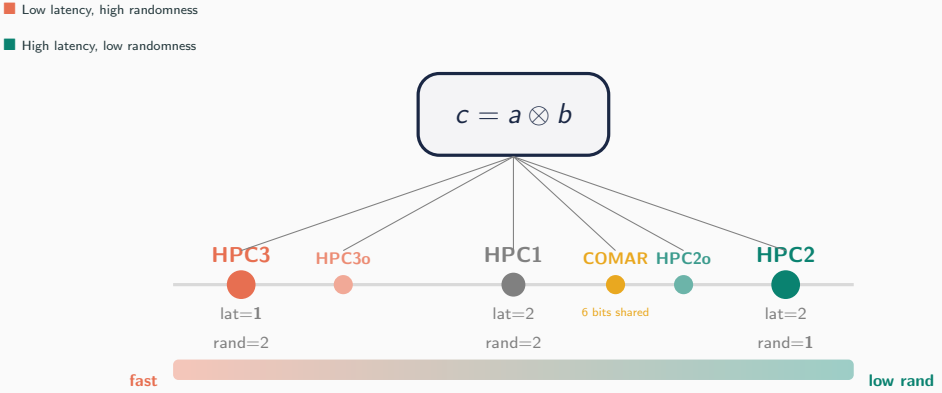
DOM AND gadget: $c = a \otimes b$



Removing a **teal** register breaks the masking proof: an attacker probing after the $\oplus$ can see a glitch-recombined function of both shares.

Removing an **amber** register only desynchronizes the pipeline – TETRIS's register-balancing step (Sec. 11) is exactly about minimizing these.

The Trade-off: Same Operation, Different Costs



A circuit has many AND operations across multiple levels.

TETRIS assigns the optimal gadget to each level

Problem & Motivation

Masking: Essential but Expensive

The cost of masking:

- **Randomness** – fresh bits for re-masking
- **Latency** – registers to prevent glitches
- **Area** – scales with pipeline registers

Randomness and latency trade off.

No tool exploits the *structure* of the trade-off

Prior Work

- **AGEMA** [KMMS21]:
  uniform gadgets, no optimization
- **COMPRESS** [CGM+24]:
  global constraint solver, heavy computation

What's missing:

- No tool formulates the *dual* problems:
  - min randomness given latency
  - min latency given randomness

The Randomness-latency trade-off in masking gadgets

d	Gadget	Lat.(in1)	Lat.(in2)	Rand.	Area (wo rand)	#regs
2	hpc1	2	1	2	50.7	6
2	hpc2o	2	1	1	60.64	10
2	hpc3o	1	1	2	43.09	6
2	hpc2	2	1	1	79.26	14
2	hpc3	1	1	2	55.32	8
3	hpc1	2	1	5	127	12
3	hpc2o	2	1	3	163.85	27
3	hpc3o	1	1	6	115.71	15
3	hpc2	2	1	3	193.9	33
3	hpc3	1	1	6	134.06	18
4	hpc1	2	1	10	217.3	20
4	hpc2o	2	1	6	315.47	52
4	hpc3o	1	1	12	222.37	28
4	hpc2	2	1	6	358.56	60
4	hpc3	1	1	12	246.84	32
5	hpc1	2	1	15	325	30
5	hpc2o	2	1	10	515.50	85
5	hpc3o	1	1	20	363.09	45
5	hpc2	2	1	10	573.23	95
5	hpc3	1	1	20	393.68	50

Area (wo rand): Area using Yosys 0.33 + Nangate 45nm without randomness generation area. #regs indicates register count from TETRIS.

The Trade-off is Structured

Gadget library (1st order, 2 shares):

Gadget	Lat.	Rand.	Area
HPC1	2	2	large
HPC3	1	2	small
HPC2	2	1	medium
HPC2o	2	1	small
HPC3o	1	2	small
COMAR	2	6*	large

*shared across all instances

Low latency, high randomness

High latency, low randomness

Key observation:

Different gadgets for the *same* non-linear operation offer different latency-randomness points.

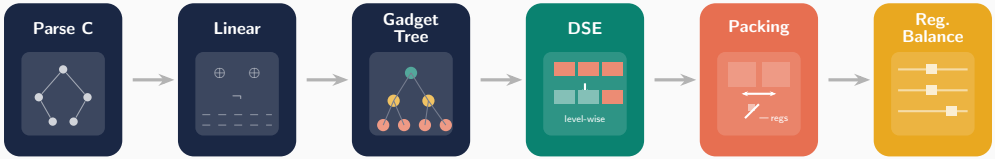
Composing them via **PINI gadgets** yields a *gadget tree* where:

- Each **level** can be assigned independently
- A **buffer** (leftover latency or randomness) can be redistributed across levels
- **Greedy** level-by-level assignment is globally optimal

$\Rightarrow$ **No heavy global optimization needed.**

TETRIS Flow

TETRIS: Unmasked C → Secure Masked RTL



- Input: **unmasked C** + user constraint
- Operates at the **behavioral level**
- Output: **PINI-secure pipelined Verilog**
- Integrated with **MaskedHLS** [STK24]

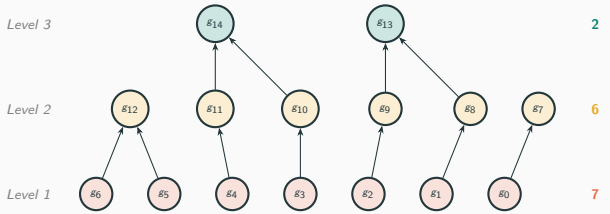
DSE Algorithms

From AST to Gadget Tree

Steps 1–3: Build the gadget tree

1. Parse C → AST
2. Mark Toffoli candidates (AND-XOR pairs)
3. Assign linear gates share-by-share
4. Abstract non-linear nodes into a **gadget tree**

Node = AND/Toffoli placeholder.
Level = nodes at the same depth.



Boyar–Peralta excerpt – node count per level drives the optimization.

Why Registers = Area (Motivation)

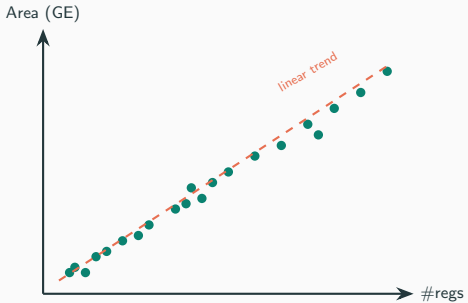
Empirical observation:

The area of a masked design increases **linearly** with pipeline register count, independent of gadget type or masking order.

$\Rightarrow$ **Minimizing registers $\approx$ minimizing area.**

Two sources of registers:

1. **Mandatory** – required by the gadget for masking security
2. **Balancing** – inserted to synchronize parallel paths in pipelined designs

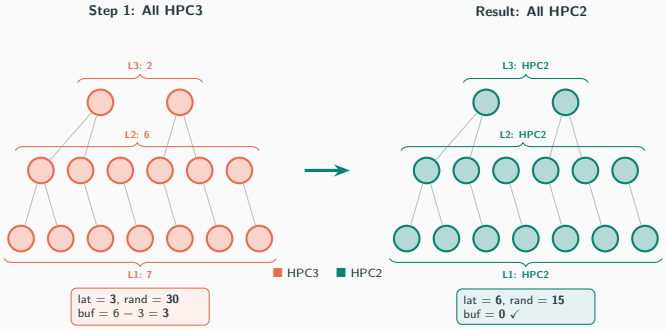


All gadgets, all orders
(from Table in Figure 2)

TETRIS minimizes both: DSE $\rightarrow$ fewer mandatory regs; MaskedHLS $\rightarrow$ optimal balancing regs

MRLC: Step-by-Step on a Gadget Tree

Setup: $d-1=1$, $target_latency=6$, HPC3 (lat=1, rand=2), HPC2 (lat=2, rand=1)



MRLC: How Buffer Latency is Consumed Level-by-Level

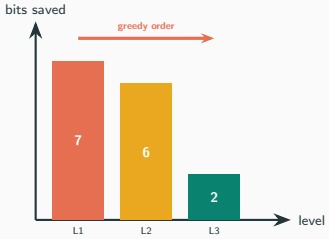
Iter	Action	Gadget tree (L1 — L2 — L3)	Rand / Buf
Init	All ← HPC3		**30** / buf=**3**
1	Swap **L1** (7 nodes) max-heap pop		**23** (−7) / buf=**2**
2	Swap **L2** (6 nodes) next in heap		**17** (−6) / buf=**1**
3	Swap **L3** (2 nodes) last in heap		**15** (−2) / buf=**0** ✓

HPC3 (lat=1, rand=2) HPC2 (lat=2, rand=1)

Optimal: randomness halved (30 → 15), latency = target (6), buffer fully consumed

Why Biggest Levels First?

Each buffer buys |level| bits.



With buf = 2:

Choice	Saved
L1 + L2	**13**
L1 + L3	9
L2 + L3	8

Each level is an "item":

- Weight: 1 unit of latency (all equal)
- Value: |level| bits of randomness

Equal weights ⇒ sort by value ⇒ greedy is optimal.

MLRC is the dual:

- Budget: randomness bits
- Cost per level: |level| bits
- Gain: 1 unit of latency

⇒ smallest levels first (cheapest latency gain)

Complexity: $\mathcal{O}(n \log n)$

MLRC: The Dual in Action

Setup: all HPC2, *target_rand* = 23, design_rand = 15, buf_rand = 8

Iter	Action	Gadget tree (L1 — L2 — L3)	Lat / Buf_rand
Init	All ← HPC2		lat=**6** / buf=**8**
1	Swap **L3** (2 nodes) min-heap pop costs **2 bits**		lat=**5** (−1) / buf=**6**
2	Swap **L2** (6 nodes) costs **6 bits**		lat=**4** (−1) / buf=**0**
–	**L1** (7 nodes) costs 7 > buf 0 **skip**		lat=4 / buf=**0**

2 levels reduced (6 → 4 cycles), L1 stays HPC2 (too expensive), rand = 15 + 8 = 23 ≤ target ✓

Complexity of Greedy MRLC and MLRC

Both algorithms share the same skeleton: build the gadget tree once, build a heap over its $L \leq n$ levels, then sweep the buffer.

MRLC (min-randomness)

Build gadget tree (n nodes)	$O(n)$
Max-heap of level sizes ($\leq n$ entries)	$O(n \log n)$
Sweep *buffer_latency* $\leq L \leq n$ levels, pop & swap	$O(n)$

$$T(n) = O(n) + O(n \log n) + O(n) = O(n \log n)$$

MLRC (min-latency)

Build gadget tree (n nodes)	$O(n)$
Min-heap of level sizes ($\leq n$ entries)	$O(n \log n)$
Sweep *buffer_randomness* $\leq n$, replace gadget, $O(1)$ each	$O(n)$

$$T(n) = O(n) + O(n \log n) = O(n \log n)$$

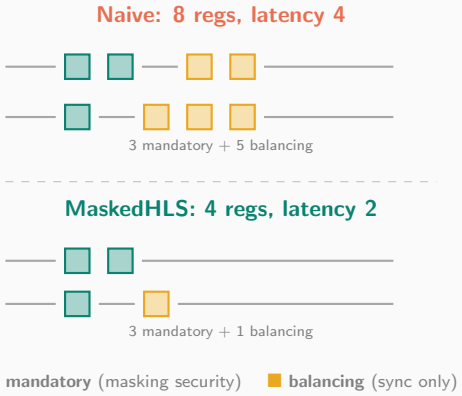
Both duals run in $\mathcal{O}(n \log n)$ – dominated by heap construction, not the greedy sweep itself.

Register Balancing via MaskedHLS

Problem: unequal register counts on parallel paths after gadget assignment ⇒ pipelining needs *balancing*.

- **Naive:** balance at gadget level
  ⇒ **8 regs, latency 4**
- **MaskedHLS:** keep only mandatory regs, balance globally
  ⇒ **4 regs, latency 2**

Key insight: pipeline registers in gadget specs aren't required for security – MaskedHLS strips them and reinserts only the minimum, **preserving TETRIS's latency**.



Asymmetric Gadget Packing

Why Asymmetry Matters

Not every gadget instance needs the same internal registers.

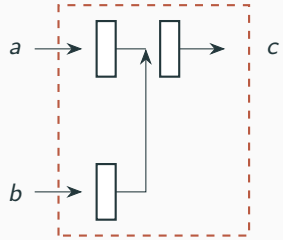
- A **symmetric** gadget registers *both* of its output shares at the same depth.
- An **asymmetric** gadget instance can drop a register on one output share *if* the consuming gate downstream doesn't need it yet – as long as PINI composability is preserved.

$\Rightarrow$ **Transforms 1–4** flip which output is delayed, letting adjacent gadgets share slack instead of each paying for its own balancing register.

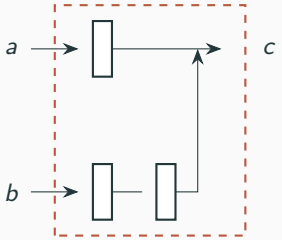
The next four slides walk through this register-by-register, showing latency, randomness, and balancing-register count at each step.

Symmetric vs. Asymmetric Gadget Instances

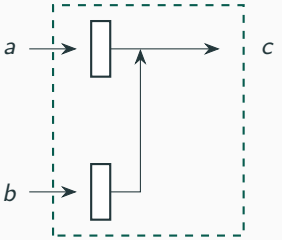
A gadget instance need not register both output shares at the same depth.



(a) asymmetric default



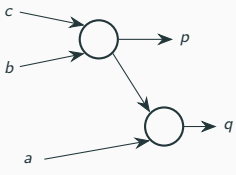
(b) asymmetric flipped



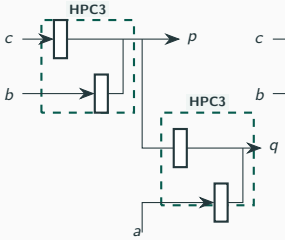
(c) symmetric default

In (a)/(b) one share path carries an extra register relative to the other (**asymmetric**); in (c) both paths register at the same depth (**symmetric**). Which share is delayed – (a) vs. (b) – is the degree of freedom Transforms 1–4 exploit.

Asymmetric Packing: Example on a Chained Gadget Pair

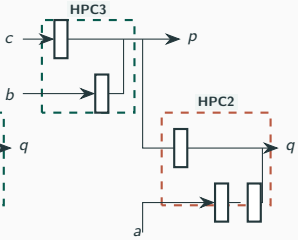


(a) AST fragment:
$p = g_1(c, b)$, $q = g_2(p, a)$



latency $= 2$, *randomness* $= 4$

(b) naive: two HPC3s

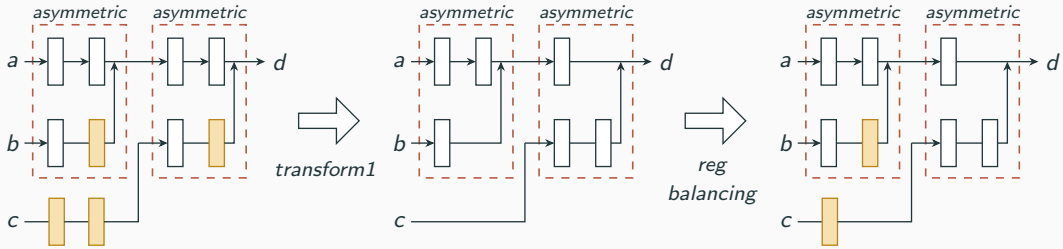


latency $= 2$, *randomness* $= 3$

(c) asymmetric: HPC3 + HPC2

Both packings keep **latency** $= 2$. Once p feeds q's gadget, its second input (a) can register *asymmetrically* in an HPC2 – saving **1 random bit** over using HPC3 again, at no extra cycle.

Gadget Transformations and Register Balancing (1/4)



latency = 4
no of balancing regs = 4
randomness(total) = 2

(a)

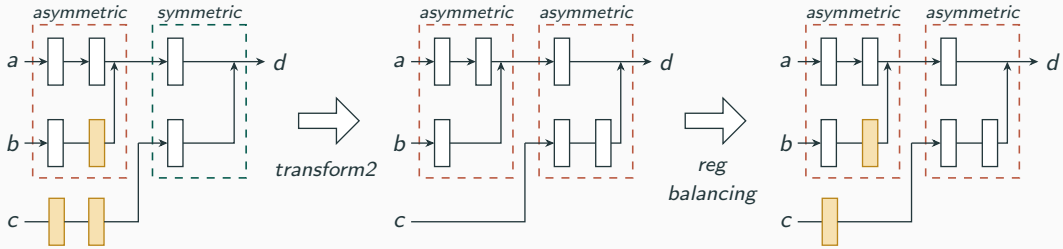
latency = 3
randomness(total) = 2

(b)

latency = 3
no of balancing regs = 2
randomness(total) = 2

(c)

Gadget Transformations and Register Balancing (2/4)



latency = 3
no of balancing regs = 3
randomness(total) = 3

(d)

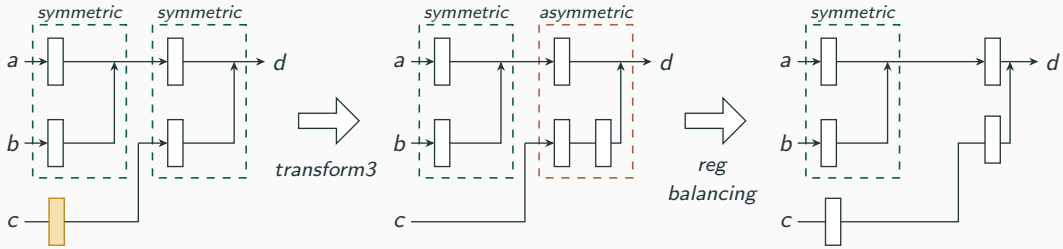
latency = 3
randomness(total) = 2

(e)

latency = 3
no of balancing regs = 2
randomness(total) = 2

(f)

Gadget Transformations and Register Balancing (3/4)



latency = 2
no of balancing regs = 1
randomness(total) = 4

(g)

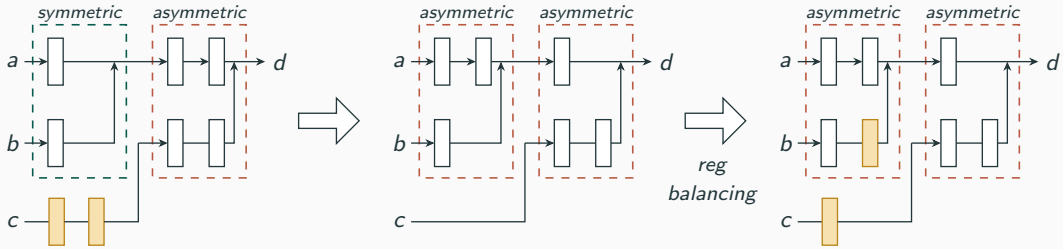
latency = 2
randomness(total) = 3

(h)

latency = 2
no of balancing regs = 0
randomness(total) = 3

(i)

Gadget Transformations and Register Balancing (4/4)



latency = 3
no of balancing regs = 2
randomness(total) = 3

(j)

latency = 3
randomness(total) = 2

(k)

latency = 3
no of balancing regs = 2
randomness(total) = 2

(l)

COMAR and randomness re-use

Using COMAR [KM22]: Randomness Reuse Requires DP

Why greedy fails:

COMAR's 6 bits are *shared* across all instances, but at any single level 6 > HPC2/HPC3's cost, so greedy never picks it – even with a `COMAR_used` flag, since levels are evaluated independently and greedy never makes the initial "investment."

But: at n levels, COMAR costs $6 + \underbrace{0 + \cdots + 0}_{n-1} = 6$ total.

Optimal substructure still holds: the best assignment through level i depends only on latency used and whether COMAR has been used.

DP formulation:

State $\mathrm{dp}[i][j][k]$: level i, latency used j, COMAR used $k \in \{0, 1\}$.
Non-COMAR gadget g:

$$\mathrm{dp}[i][j][k] = \mathrm{dp}[i-1][j-g.\ell][k] + g.r$$

COMAR, first use:

$$\mathrm{dp}[i][j][1] = \mathrm{dp}[i-1][j-2][0] + 6$$

COMAR, reused:

$$\mathrm{dp}[i][j][1] = \mathrm{dp}[i-1][j-2][1]$$

$\mathcal{O}(L \cdot T \cdot |\mathcal{G}|)$ – 1st-order only.

Results

Benchmark Details

S-box (reference)	AND levels	Total AND gates
AES - Boyar and Peralta's S-box [BP12]	4	34
AES - Canright's S-box [Can05]	4	36
SKINNY 8 bit S-box [BJK$^+$16]	4	8

Table 1: Comparison of different S-box designs.

MRLC comparison with COMPRESS for AES Boyar and Peralta's S-box

$d-1$	Latency	Design	#Random	Area (kGE) w/o rand	Area (kGE) w/ rand	Time (sec)	Speedup
1	4	Tetris	46	2.83	4.64	0.063	—
		Comp: Base		3.53	5.34	1.01	15.9×
		Comp: Sep		3.24	5.05	1.63	25.6×
		Comp: Opt		**2.78**	**4.59**	2.35	**36.9**×
2	4	Tetris	138	**6.68**	**12.11**	0.085	—
		Comp: Base		7.91	13.34	1.12	13.2×
		Comp: Sep		7.47	12.9	1.68	19.8×
		Comp: Opt		6.79	12.22	2.28	**26.8**×
3	4	Tetris	276	**12.16**	**23.03**	0.08	—
		Comp: Base		14.03	24.89	1.11	13.9×
		Comp: Sep		13.44	24.3	1.76	22.0×
		Comp: Opt		12.96	23.82	2.27	**28.4**×
4	4	Tetris	460	**19.77**	**37.89**	0.12	—
		Comp: Base		21.89	40	1.1	9.2×
		Comp: Sep		21.15	39.26	1.64	13.7×
		Comp: Opt		20.45	38.55	2.28	**19.0**×

Table 2: MRLC comparison with COMPRESS: Boyar-Peralta's S-box Results

MRLC comparison with COMPRESS for AES Boyar and Peralta's S-box

$d-1$	Latency	Design	#Random	Area (kGE) w/o rand	Area (kGE) w/ rand	Time (sec)	Speedup
1	6	Tetris	34	3.85	5.19	0.055	—
		Tetris: w/ comar	31	3.71	4.93	0.053	—
		Comp: Base	34	4.06	5.4	2.53	46.0×
		Comp: Sep	34	3.77	5.11	8.69	158.0×
		Comp: Opt	34	**3.34**	**4.67**	17.43	**316.9**×
2	6	Tetris	102	9.04	13.05	0.14	—
		Comp: Base		9.0	13.01	2.41	17.2×
		Comp: Sep		8.56	12.57	8.58	61.3×
		Comp: Opt		**7.92**	**11.94**	14.79	**105.6**×
3	6	Tetris	204	16.4	24.43	0.94	—
		Comp: Base		15.87	23.9	2.34	2.5×
		Comp: Sep		15.28	23.9	8.25	8.8×
		Comp: Opt		**14.88**	**22.91**	32.28	**34.3**×
4	6	Tetris	340	25.9	39.29	0.98	—
		Comp: Base		24.69	38.07	2.33	2.4×
		Comp: Sep		23.95	37.33	6.88	7.0×
		Comp: Opt		**23.37**	**36.65**	13.83	**14.1**×

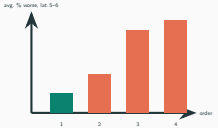
Table 3: MRLC comparison with COMPRESS: Boyar-Peralta's S-box Results

Where TETRIS's Speed Costs Area: vs. COMPRESS:Opt

Order $d-1$	lat. 4	lat. 5	lat. 6
1	-1.1%	$+5.9\%$[1]	$+5.6\%$[1]
2	-0.9%	$+4.2\%$	$+9.3\%$
3	-3.3%	$+1.6\%$	$+6.6\%$
4	-1.7%	$+1.8\%$	$+7.2\%$

teal = TETRIS smaller; coral = TETRIS larger
(area w/ randomness, Boyar–Peralta S-box)

[1] with COMAR's shared randomness – plain TETRIS is $+6.4\%$ (lat. 5) / $+11.1\%$ (lat. 6).



The pattern:

- At the **tight budget** (latency = 4), TETRIS **matches or beats** COMPRESS:Opt on area.
- Slack the budget to latency 5–6 and TETRIS falls behind by **2–11%**, growing with both latency and order.
- **COMAR** claws back roughly half the gap at order 1 ($+11.1\% \rightarrow +5.6\%$).

Trade-off: that area buys **DSE in <1 ms** vs. Comp:Opt's 2–32 s solver runs – **13–317×** faster across this sweep.

MRLC Results: Summary

Boyar–Peralta	**Canright**	**SKINNY**
34 AND, 4 levels	**36 AND, 4 levels**	**8 AND, 4 levels**
Randomness: = **COMPRESS**	Randomness: = **COMPRESS**	Randomness: = **COMPRESS**
Speedup: **up to 317×**	w/ COMAR: **better**	Speedup: **up to 52×**
Area: **comparable**	Area: **comparable**	Area: **21–29% less**

DSE $<$ 1 ms | **Orders 1–4** | **Gadgets: HPC1/2/3, HPC2o/3o, COMAR**

Note: COMPRESS uses pre-optimized sub-circuits for the Canright's S-box (e.g., randomness-sharing via *G*16_*mult*).

TETRIS performs gadget-level DSE only – these optimizations can be layered on top.

MLRC Results

Table 4: MLRC Results: AES Boyar-Peralta's S-box for each Latency and Security Order

Security Order	Latency	Target_Randomness	Randomness_used	Area (kGE) (w/o rand)	Time (sec)
1	4	57	46	2.81	0.167
		58	46	2.81	0.166
	5	43	37	3.35	0.094
		44	37	3.35	0.092
	6	36	36	3.75	0.061
		37	37	3.74	0.064
	7	34	34	4.20	0.058
		35	35	4.16	0.066
2	4	172	138	6.66	0.298
		174	138	6.66	0.292
	5	130	111	7.91	0.159
		132	111	7.91	0.315
	6	108	108	8.75	0.228
		110	110	8.68	0.118
	7	102	102	9.79	0.073
		104	104	9.67	0.078

MLRC Results

Table 5: MLRC Results: AES Boyar-Peralta's S-box for each Latency and Security Order

Security Order	Latency	Target_Randomness	Randomness_used	Area (kGE) (w/o rand)	Time (sec)
3	4	344	276	12.13	0.444
		348	276	12.13	0.467
	5	260	222	14.39	0.154
		264	222	14.39	0.154
	6	216	216	15.81	0.123
		220	219	15.66	0.106
	7	204	204	17.67	0.093
		208	207	17.42	0.088
4	4	570	460	18.57	0.879
		575	460	18.57	0.916
	5	430	370	22.03	0.305
		440	370	22.03	0.301
	6	360	360	24.13	0.155
		365	364	23.86	0.130
	7	340	340	27.00	0.144
		345	344	26.66	0.119

MLRC Results: Summary

Table 6: MLRC Overall Summary: Average DSE Performance Across S-boxes (No Prior Work Solves This Dual Problem, So No Baseline Comparison Is Possible)

S-box (#AND)	#Configs (orders × latencies)	Avg. DSE Time (ms)	Max DSE Time (ms)	Avg. Rand. Saved vs. target (%)	Max Rand. Saved vs. target (%)
Boyar–Peralta (34)	32	215.8	916.0	8.8	20.7
SKINNY (8)	38	70.1	417.0	26.6	45.5
Canright (36)	24	202.4	676.0	11.8	20.7
Overall	**94**	**153.5**	**916.0**	**16.8**	**45.5**

Rand. Saved is the reduction MLRC achieves below the user's requested `target_randomness` bound while hitting the tighter latency; #Configs counts (security order, target latency) pairs swept across Tables 4–5 for B & P and others (both target-randomness values per row averaged). All runs complete in well under 1 second.

Area comparisions for SOTA Full AES-128: 16-round Pipelined Implementation

Design	Datapath	Latency (cycles)	d=1 (kGE)	d=2 (kGE)	d=3 (kGE)	d=4 (kGE)
AGEMA-HPC2 [KMMS21]	128-bit	91	170.9	354.2	602.9	917.2
AGEMA-HPC3 [KMMS21]	128-bit	51	148.4	344.2	621.1	979.2
COMPRESS: Opt [CGM+24]	128-bit	51	85.0	190.9	–	–
TETRIS	**128-bit**	**51**	**96.1**	**321.4**	–	–

- TETRIS beats **AGEMA-HPC3** at same latency
- Lower area than **AGEMA-HPC2** at lower latency
- COMPRESS uses full-cipher register sharing
- Fair comparison: same AES wrapper

Security

Security: PINI by Construction

Proof by construction (Lemma 4):

1. **Linear gadgets:** d-PINI
   share-by-share, no refresh needed
2. **Non-linear gadgets:** d-PINI
   HPC1/2/3, Toffoli variants proven in cited works
3. **Transforms 1–4:** preserve PINI
   only flip inputs or substitute gadgets
4. **Valid gadget composition:**
   DAG of PINI gadgets $\Rightarrow$ PINI design

VERICA [RBFSG22] verification:

S-box	d=1	d=2
Boyar–Peralta	PINI ✓	PROB ✓
Canright	PINI ✓	PINI ✓
SKINNY	PINI ✓	PROB ✓

Verified *after* all transforms (Steps 1–5).

Higher-order timeouts: known BDD-based verification scalability limits, not TETRIS issues.

Conclusion

Summary

- Gadget-based masking has a **structured trade-off** that decomposes **level by level**
- **MRLC** and **MLRC**: dual greedy algorithms with provably optimal gadget assignments
- **DP formulation** for COMAR's shared-randomness special case
- **Transforms 1–4**: exploit asymmetric gadgets to reduce registers, randomness, and latency
- Results: randomness = COMPRESS, area comparable, **DSE in < 1 ms** (up to 317× faster)
- All designs **PINI-secure** by construction, verified via VERICA
- **Open source**, integrated with MaskedHLS for end-to-end C → Verilog

Thank you Questions?

References i

Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim.

The skinny family of block ciphers and its low-latency variant mantis.

In *Annual international cryptology conference*, pages 123–153. Springer, 2016.

Joan Boyar and René Peralta.

A small depth-16 circuit for the aes s-box.

In *IFIP international information security conference*, pages 287–298. Springer, 2012.

David Canright.

A very compact s-box for aes.

In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 441–455. Springer, 2005.

References ii



Gaëtan Cassiers, Barbara Gigerl, Stefan Mangard, Charles Momin, and Rishub Nagpal.

Compress: Generate small and fast masked pipelined circuits.

IACR Transactions on Cryptographic Hardware and Embedded Systems, 2024(3):500–529, 2024.



David Knichel and Amir Moradi.

Composable gadgets with reused fresh masks: First-order probing-secure hardware circuits with only 6 fresh masks.

IACR Transactions on Cryptographic Hardware and Embedded Systems, 2022(3):114–140, Jun. 2022.

References iii



David Knichel, Amir Moradi, Nicolai Müller, and Pascal Sasdrich.

Automated generation of masked hardware.

IACR Transactions on Cryptographic Hardware and Embedded Systems, 2022(1):589–629, Nov. 2021.



Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu.

Verica-verification of combined attacks: Automated formal verification of security against simultaneous information leakage and tampering.

IACR Transactions on Cryptographic Hardware and Embedded Systems, pages 255–284, 2022.

References iv



Nilotpola Sarma, Anuj Singh Thakur, and Chandan Karfa.

Maskedhls: Domain-specific high-level synthesis of masked cryptographic designs.

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 43(11):3973–3984, 2024.